



Pokerbots 2026

Lecture 7: Neural Networks and Optimization

Sponsors



CITADEL | CITADEL Securities





Announcements

Hackathon Tonight!

- Wednesday night (1/14)
- 32-044 at 7pm - LATE
- Show up and grind
- Dinner provided
- Snacks, fun, and games
- Prizes for challenges and those that stick around
- Details will be announced on Piazza

Snack/Drink Recs?

pkr.bot/snack

Make-up Progress Reports due **TODAY**

- As per syllabus, if your team missed the Week 1 bot submission deadline, a progress report is required to earn credit
- Reports are half a page and must list team name and members
- Submit via **Canvas** by 11:59pm Wednesday (Today)
 - Deadline pushed back to tonight due to late announcement

Week 2 Bot Deadline

- Second pokerbot submission due Friday 1/16, 11:59PM EST on scrimmage server
- Mini-tournament 2 will occur shortly after

No Class Thursday

Cancelled in light of Hackathon the night before

No Class Monday

Happy MLK Day!



Giveaways!

Birthday Game

- Guess the average Pokerbots exec team member age
 - Enter this number in days
- Feel free to use decimals
- Winner is closest guess to the true answer
 - Ties broken by earliest submission time
- Prize: JBL Flip 5

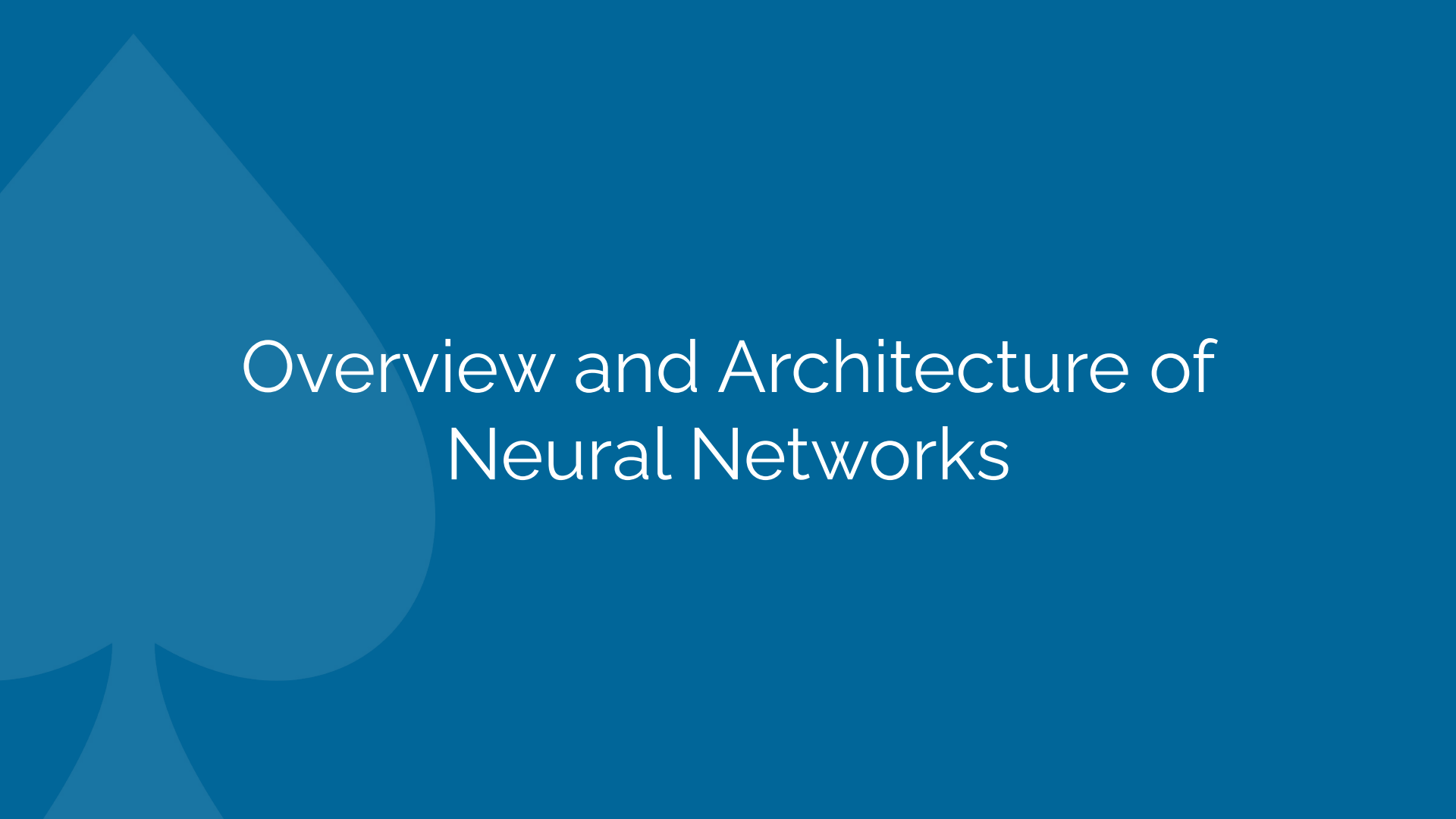
pkr.bot/bday



Neural Networks & Optimization

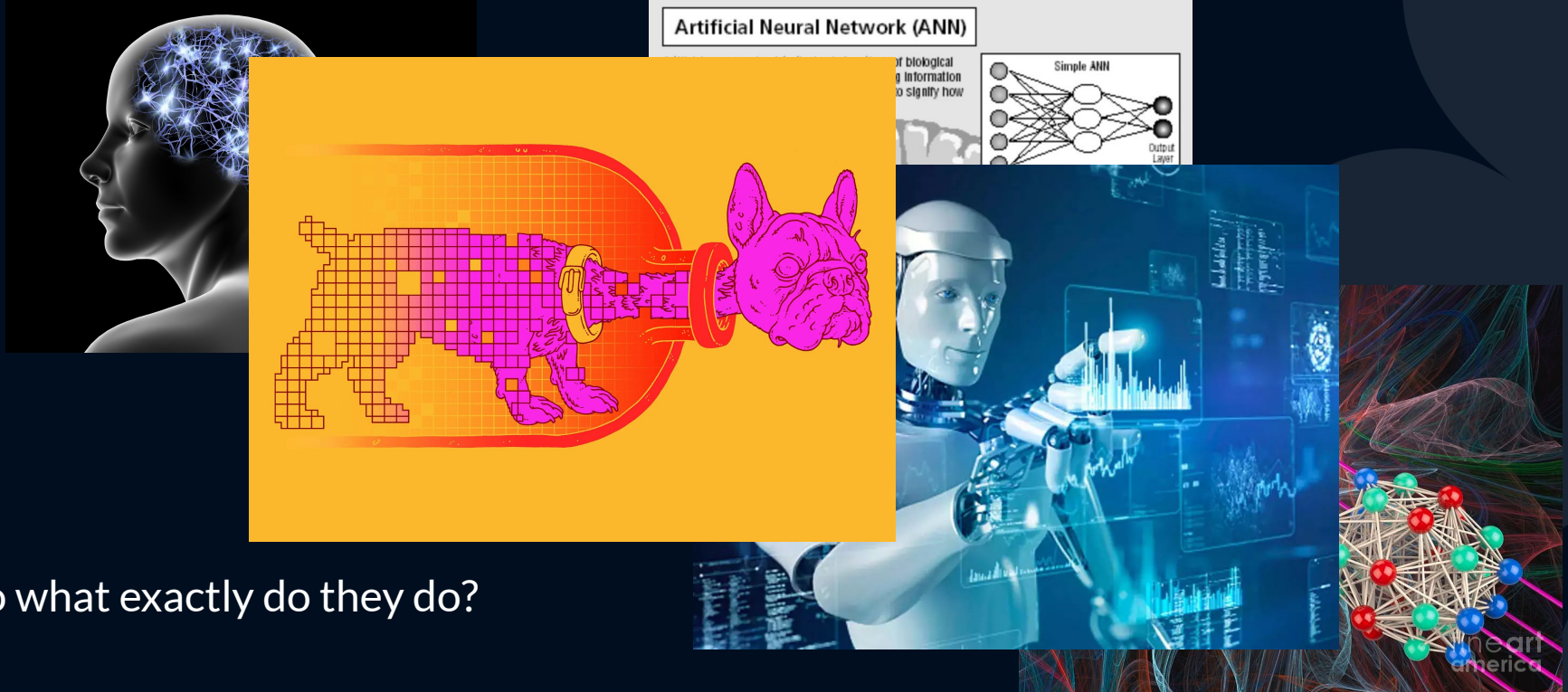
Agenda

- Overview and Architecture of Neural Networks
- ML Optimization and Training Algorithms
- Neural Network Applications and Considerations
- Methodical Design of Neural Networks

The background is a solid dark blue. On the left side, there are several overlapping, semi-transparent shapes in lighter shades of blue. These shapes include a large triangle pointing upwards and a large circle, creating a layered, abstract effect.

Overview and Architecture of Neural Networks

Neural Networks seem to be all the hype...

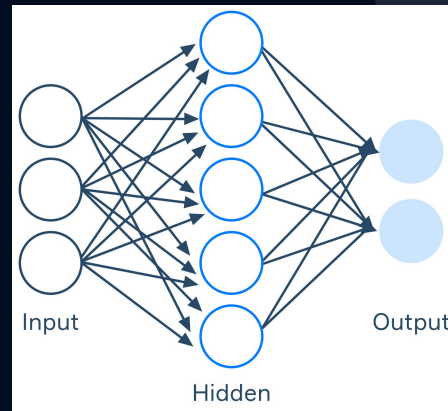


So what exactly do they do?

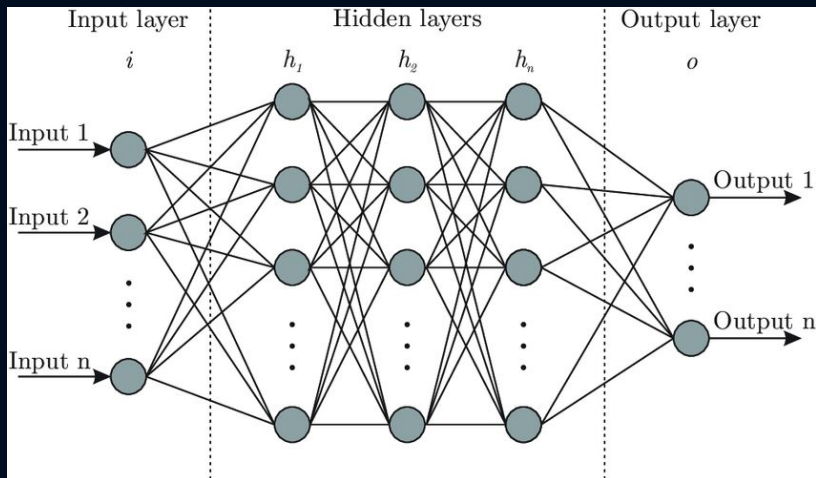
Another parameterized model class

Recall: “template for a machine learning algorithm”

- We specify such a template using *hyperparameters*
- then tweak settings within the template called *parameters*
- they modify the behavior of our *algorithm*
- which is just a function that takes in input and gives us output
- ideally our outputs are good



Neural Network Anatomy

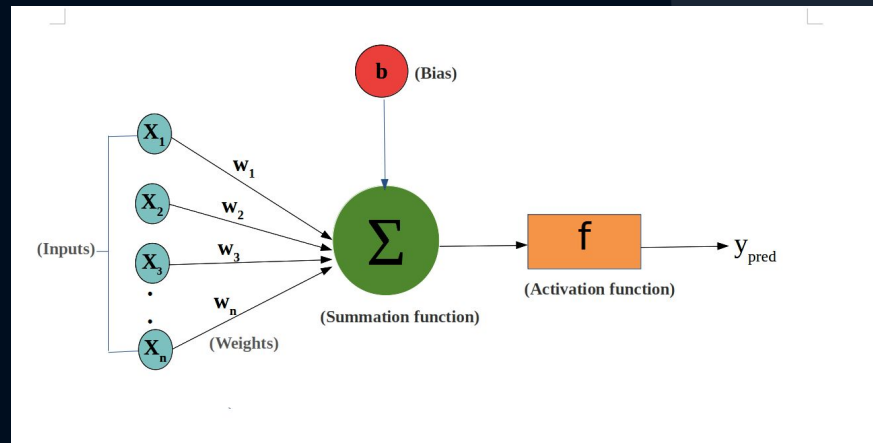


- Comprised of Layers
 - Input layer and output layer
 - “Hidden layers”: every layer in between
 - How many is up to us!
- Each layer is a set of values, or “neurons”
 - Input layer size = dimension of input
 - Output layer size = dimension of output
 - Hidden layer sizes are up to us!
- Evaluation starts at input layer
- The neurons in each layer are calculated using those from the previous layer
- Resulting values in the output layer is the output of the model

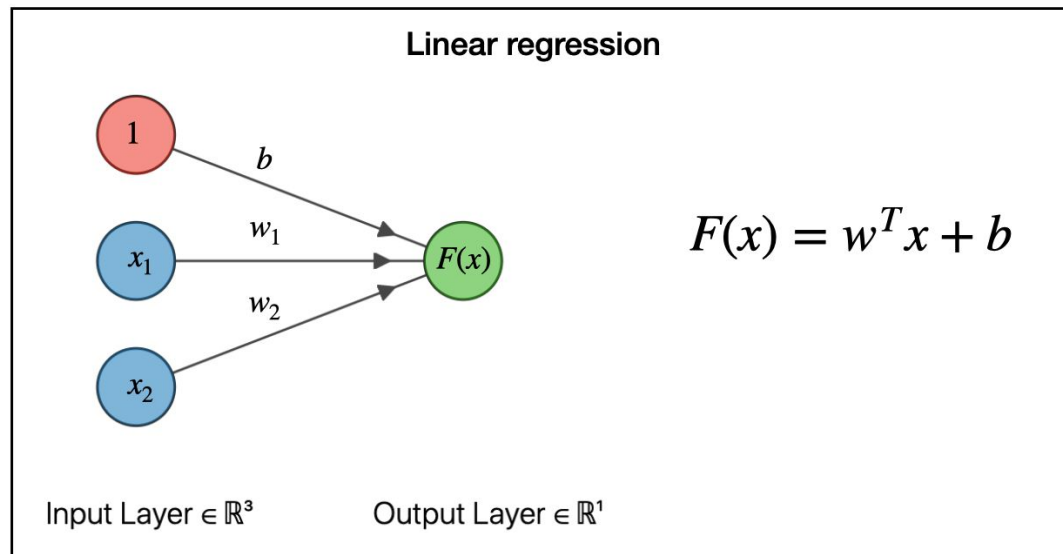
Evaluation of Layers

Each neuron is a transformed weighted sum of the previous layer, i.e.

- All the neurons of the previous layer are assigned *weights*.
- They are multiplied by their weights and summed, often with a *bias* value.
- This linear combination is then passed through an *activation function*



Linear Regression as a Neural Net



$$F(x) = w_1 \cdot x_1 + w_2 \cdot x_2 + 1 \cdot b$$

Weights and Biases

There is a unique set of weights and biases for every pair of consecutive layers:

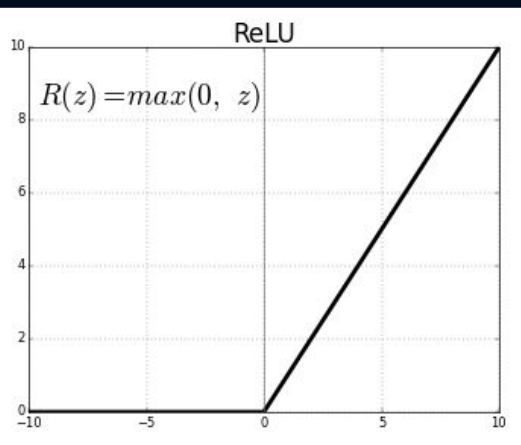
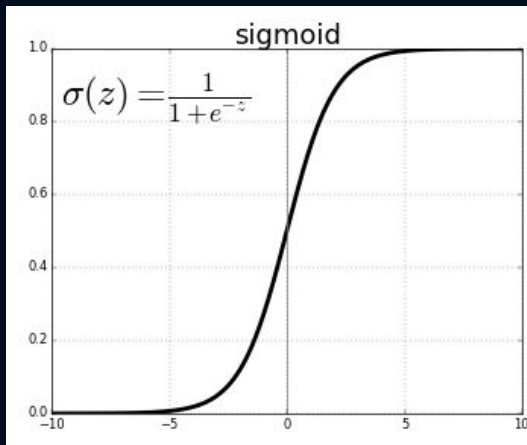
- One weight for each pair of neurons across the two layers
- One bias for each neuron in the next layer

These values comprise the *parameters* of the neural network.

Our usual goal (e.g. supervised learning context) is to find settings for these parameters such that the neural network behaves well and produces useful outputs

Activation function

- Applied to linear combinations to produce new neuron values.
- The function itself is another thing that's up to us!
- Biggest rule is that it cannot be a linear function
- We also generally want it to be monotonic (ex. Always increasing)
- Common choices: sigmoid, ReLU (Rectified Linear Unit)



All together

First step: choose *hyperparameters*, aka design neural network template

- Number of layers and neurons in each
- Activation function for each layer

This first step usually involves the task at hand (dataset size, input/output types, etc.)

Next step: choose *parameters*, aka fill in the template

- weights and biases

How do we find the right parameters?

ML Optimization and Training

General Parameterization of Functions

- A neural network is an example of a general parameterized function, which can be expressed as $f(x; \theta)$, where θ represents the current setting of parameters, and x is the input.
- In a machine learning setting we would like to choose θ such that the loss resulting from the output $y = f(x; \theta)$ is as small as possible
- Finding a good θ will involve trying different values and evaluating their loss on our training data to see if it worked

Naive Idea: parameter tweaking

- Start with some parameter values
- Make a bunch of copies and make different very small changes to each copy
- Evaluate the original and each copy by computing the total loss and keep the best one
- Repeat

General idea here: find the best “direction” to shift the parameters in...

Gradient Descent

Use calculus to find the best direction!

This direction is a vector of updates to our parameters, given by the gradient of total loss with respect to the parameters.

Compute the gradient and take a step scaled by some factor

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

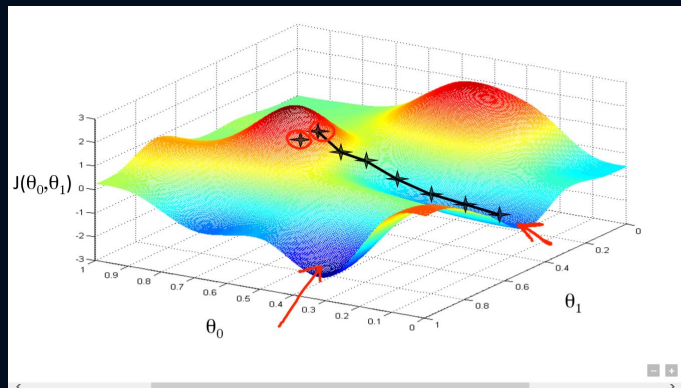
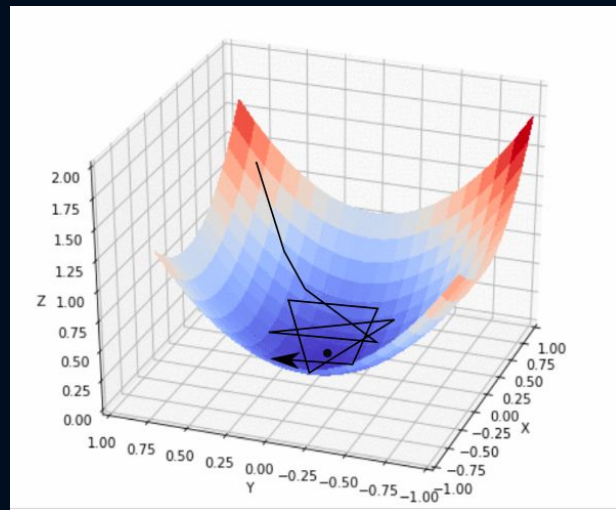
η : Learning rate (step size).

$L(\theta)$: Gradient of the loss function w.r.t. parameters.

Convex Optimization

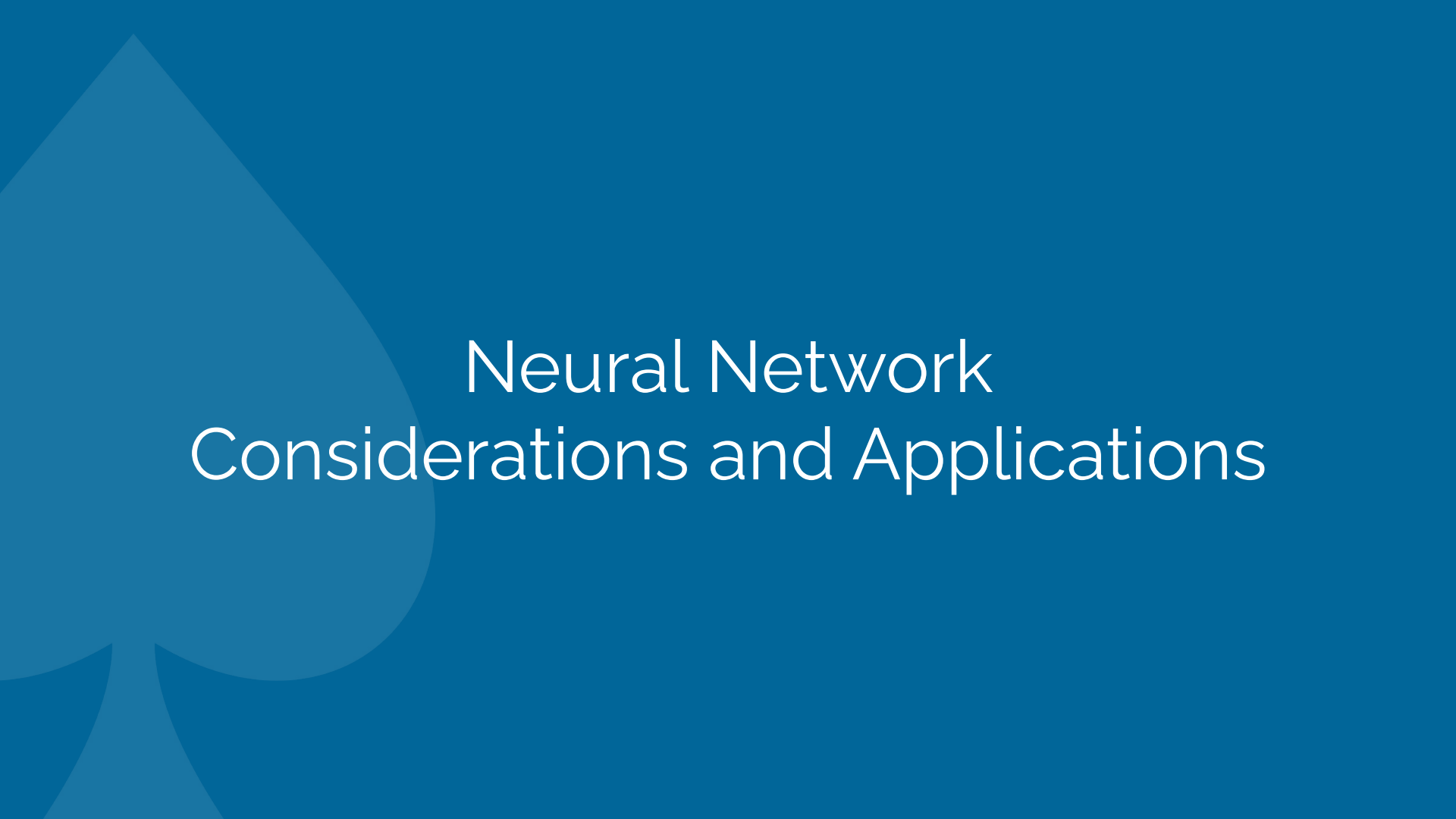
For convex functions and carefully chosen step size, this will find the very best set of parameters (global optima)

However landscape on real datasets is rarely nice and convex



Making Gradient Descent Better

- Stochastic Gradient Descent (SGD)
- Batch Optimization
- Sizing step size
- Adam
- Hypernetworks!?

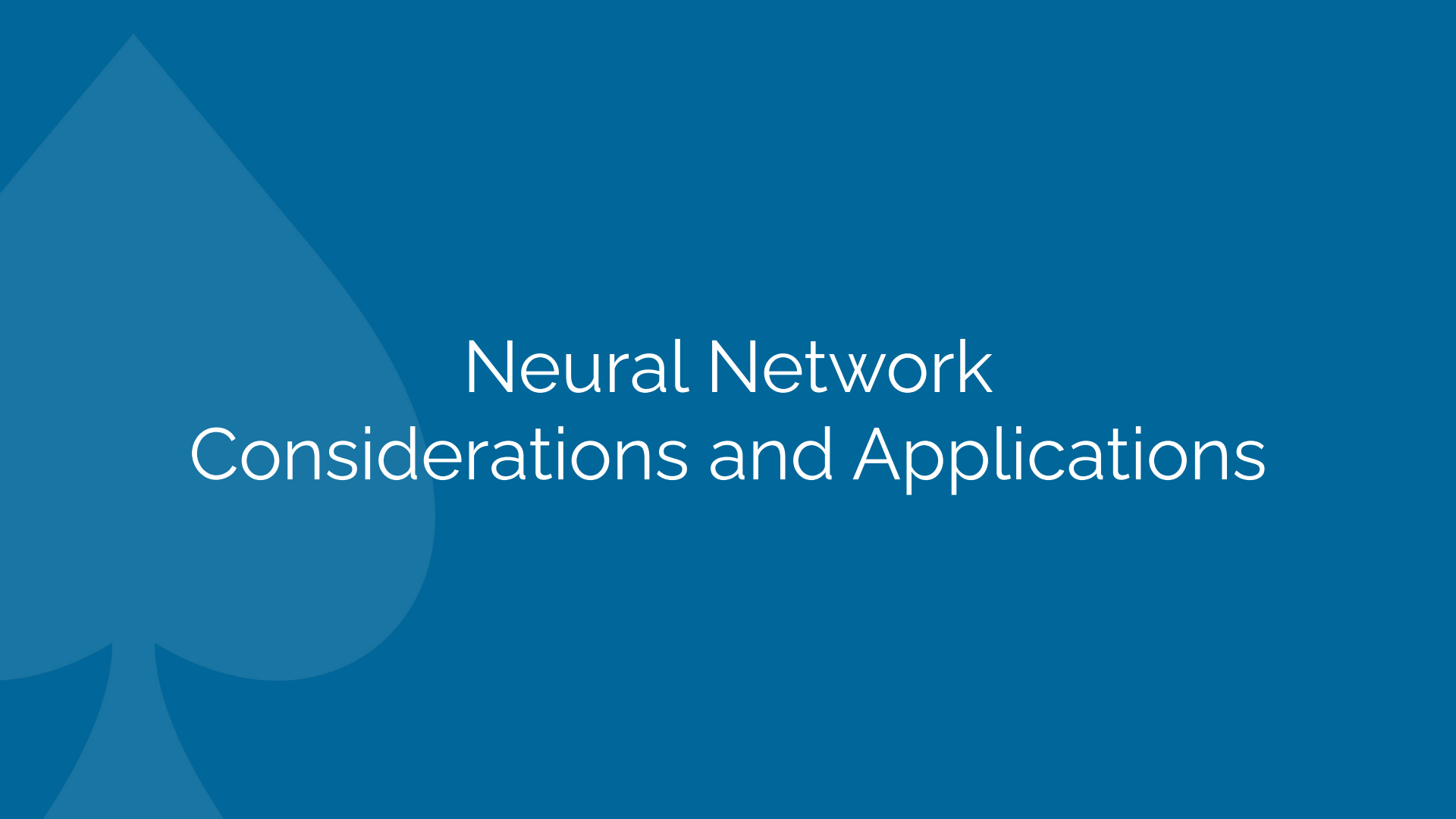
The background is a solid dark blue. On the left side, there are several overlapping, semi-transparent shapes in a lighter shade of blue. These shapes include a large triangle pointing upwards and a large circle, creating a layered, abstract effect.

Neural Network Considerations and Applications

Lunch 🤪

Leave any type of feedback at [pkr.bot/feedback!](https://pkr.bot/feedback)



The background is a solid dark blue. On the left side, there are several overlapping, semi-transparent shapes in a lighter shade of blue. These shapes include a large triangle pointing upwards and a large circle, creating a layered, abstract effect.

Neural Network Considerations and Applications

Backpropagation

An algorithm for computing gradients in a neural network using the chain rule of calculus.

- Takes advantage of the sequential computation of neural networks
- Makes use of linear algebra to simplify everything into matrix operations
- Modern libraries like PyTorch can do all the calculus for us!
- GPUs are good at matrix operations and are very helpful in training and evaluating neural networks

NNs in the wild

- Supervised Learning
 - Classification
 - Regression
 - Everything!
- Unsupervised
 - Representation Learning
 - Autoencoders
 - Language Modeling
- Reinforcement Learning
 - Approximate Value Iteration
 - Deep Q Learning
 - Policy Gradient

Why are they so useful?

- Can be modified to work in a number of settings
- “Universal function approximators”
- Extremely expressive due to combination of linear steps with nonlinear activation functions.
- Getting easier and easier to train with modern tools

Methodical Design of Neural Networks

Neural Nets Go Brrr



Is There More to It

- Universal Approximation Theorem
 - 2-layer neural network with non-linear activation function is a 'universal approximator' given sufficient hidden nodes
- So if any function can be approximated with 2 layers, should we use more?
 - Yes
- (there's more to it)

Deep Learning

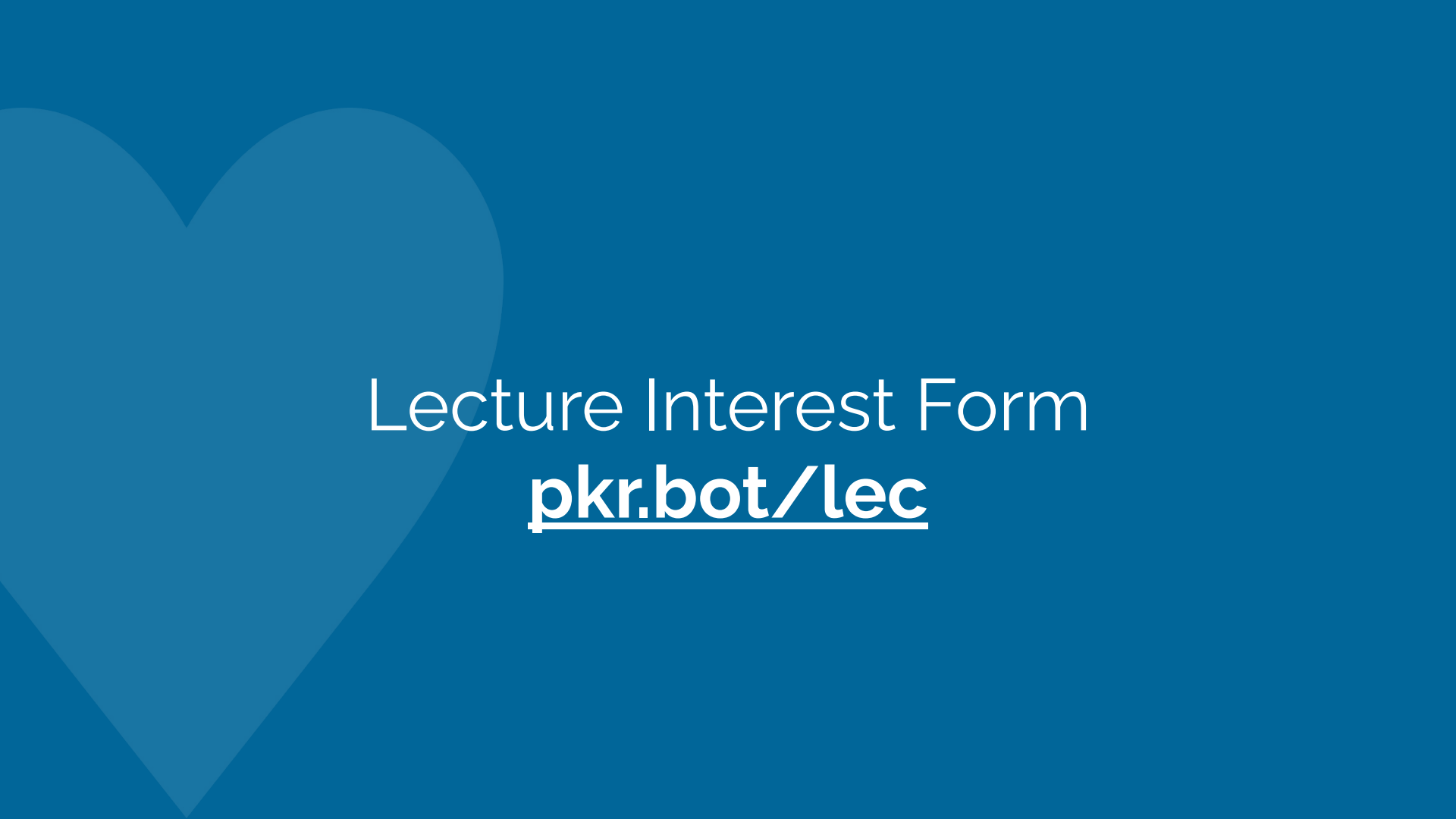
- More layers = recursive feature reuse
- Exponentially efficient representations of complex functions
- 2-layer might work, but more layers needs less total parameters!
- 'Curse of Depth'

Continuity

- In most neural nets, outputs are continuous w.r.t inputs
 - Important to consider continuity as a measure of similarity
- Example: gyroscopic input to a self-driving robot in range $[0^\circ, 360^\circ)$
 - Inputting straight into NN: bad
 - 0° and 359° are far away as numbers, close as angles
 - What can we do?
 - Split into $\sin(\text{angle})$ and $\cos(\text{angle})$: good
 - Even with wrapping around, if two angles are close, $\sin$ and $\cos$ are close

Inductive Bias (once again)

- Refresher: “the set of assumptions or preferences a model uses to **generalize**”
- Can be used to dictate the design of our neural network
 - i.e. angle continuity from last slide
- Another example:
 - Want to compress our images of cards, have 52 (one of each card)
 - Approach 1:
 - Throw an autoencoder at it, compress from 128x128x3 to just 256 dimensions
 - Approach 2:
 - Use one-hot encoding to compress to just 17 binary digits!
 - Both approaches will require training a model, but second uses **inductive bias**
 - **We** know rank and suit are separate, incorporating into model can help it learn better



Lecture Interest Form
pkr.bot/lec



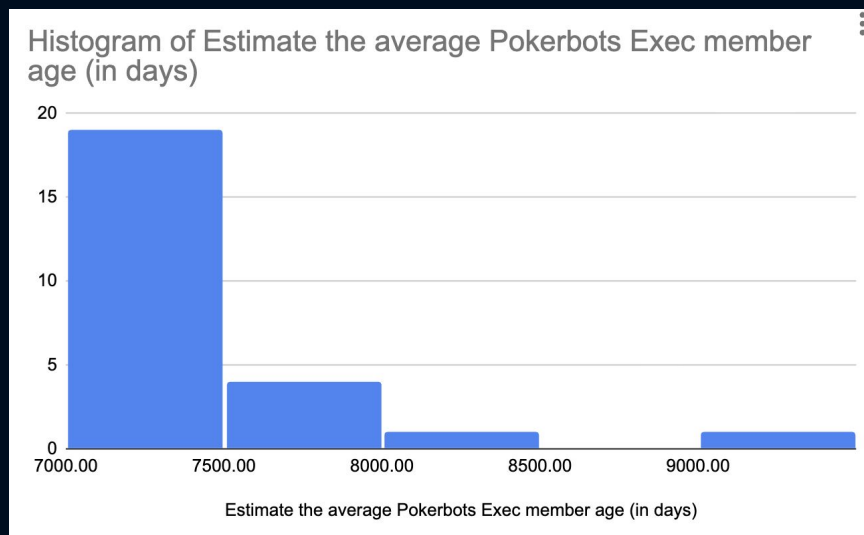
Giveaway Winner!

Bday Winner!

Answer: 7384.5 days

The average staff member is 20 years old!

Winning Guess: 7381 days!



Winner:
maxcass

Thanks for watching!

Information hub at pkr.bot/hub

Slides/notes will be posted on pkr.bot/resources

Make sure to check pkr.bot/piazza for updates

Lecture recordings at pkr.bot/panopto

Leave feedback at pkr.bot/feedback!